Lab 06

COMP1322 Programming II

Student Name: Qi Liu
Student ID: 36557854
Student Email: ql1e24@soton.ac.uk

Date: April 1, 2026

1.Practice Task: Logbook Notes

1.1 Event Handling

Event-Driven Programming: What is it, and why is it important in GUI applications?
The program waits for user clicks, typing and runs code in response. Essential for GUI apps if you wanna interact with users.

Event Loop: How JavaFX handles user interactions?
JavaFX will continuously monitor for events on the main thread and call the appropriate handler when one occurs.

Event Handlers: Different ways to implement event handling in Java (Provide one example
for each way):
- Separate class
- Anonymous inner class
- Lambda expressions (recommended)

```
// 1. Separate class
button.setOnAction(new MyHandler());

// 2. Anonymous inner class
button.setOnAction(new EventHandler<ActionEvent>() {
   public void handle(ActionEvent e) {
       System.out.println("clicked :)");
       }
});

// 3. Lambda (recommended)
button.setOnAction(e -> System.out.println("clicked :)"));
```

Common event types (provide one example each):
- ActionEvent (e.g., button clicks)
- MouseEvent (e.g., hovering, clicking)
- KeyEvent (e.g., typing)

```
button.setOnAction(e -> ...); // ActionEvent - button click
canvas.setOnMouseClicked(e -> ...); // MouseEvent - mouse click
textField.setOnKeyPressed(e -> ...); // KeyEvent - keyboard press
```

1.2 UI Controls

- **Button**: triggers an action when clicked
- **CheckBox**: select/deselect an option
- **TextField**: accepts text input from user

- **VBox / HBox / GridPane**: layout containers to arrange components

1.3 Basic Graphics in JavaFX

Canvas	Scene Graph (SVG)
Draws pixels directly	Uses node objects
Shapes can not be clicked	Nodes can edit shapes
Must manually clear to redraw	Rendering is automatic
Good for drawing	Good for standard UI

1.4 JavaFX Scene Graph

Basic Example:
- A `Group` node contains multiple UI elements like `Rectangle`, `Circle`, and `Text`(provide an example for the given statement).

```
GraphicsContext gc = canvas.getGraphicsContext2D();
gc.setFill(Color.BLUE);
gc.fillOval(100, 100, 30, 30); // blue circle
```

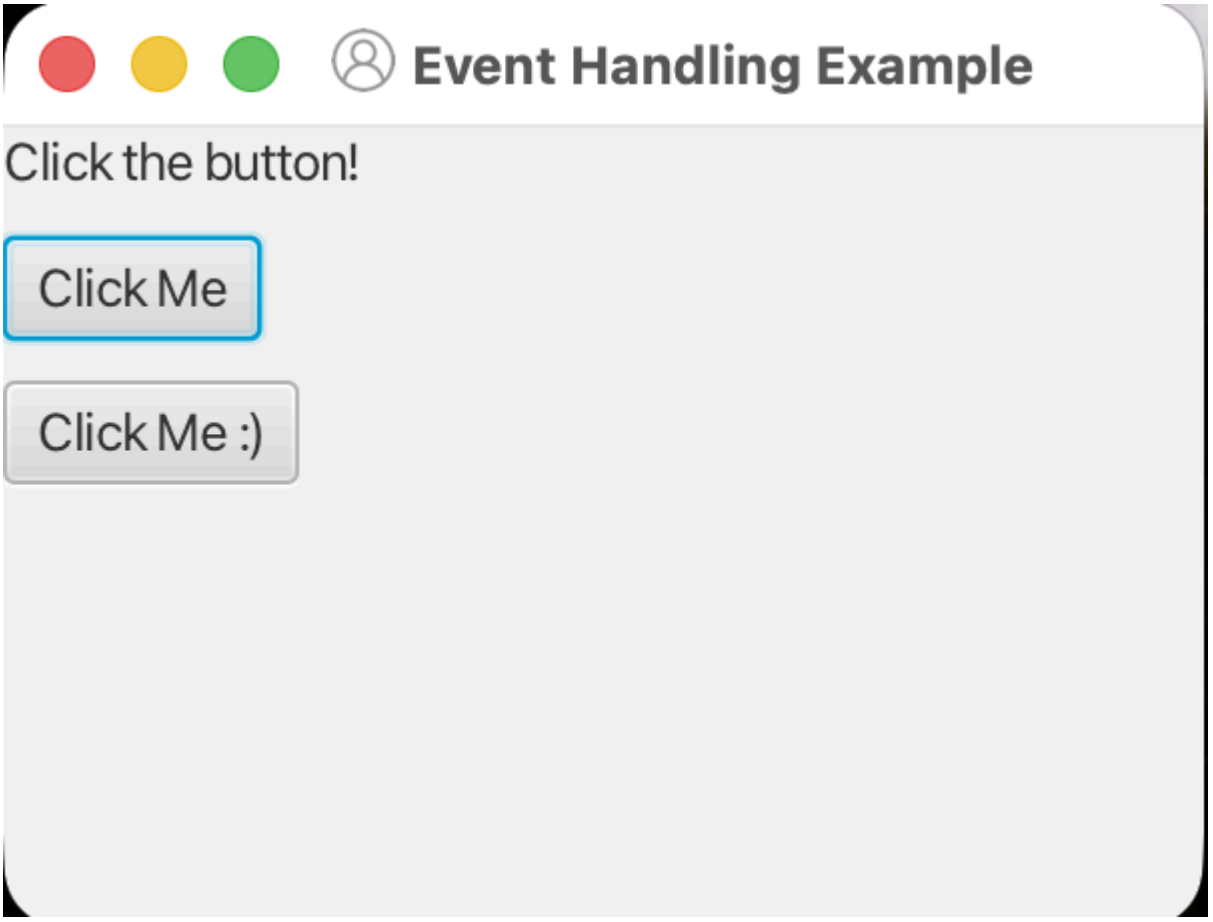
2. Programming Task

Code:

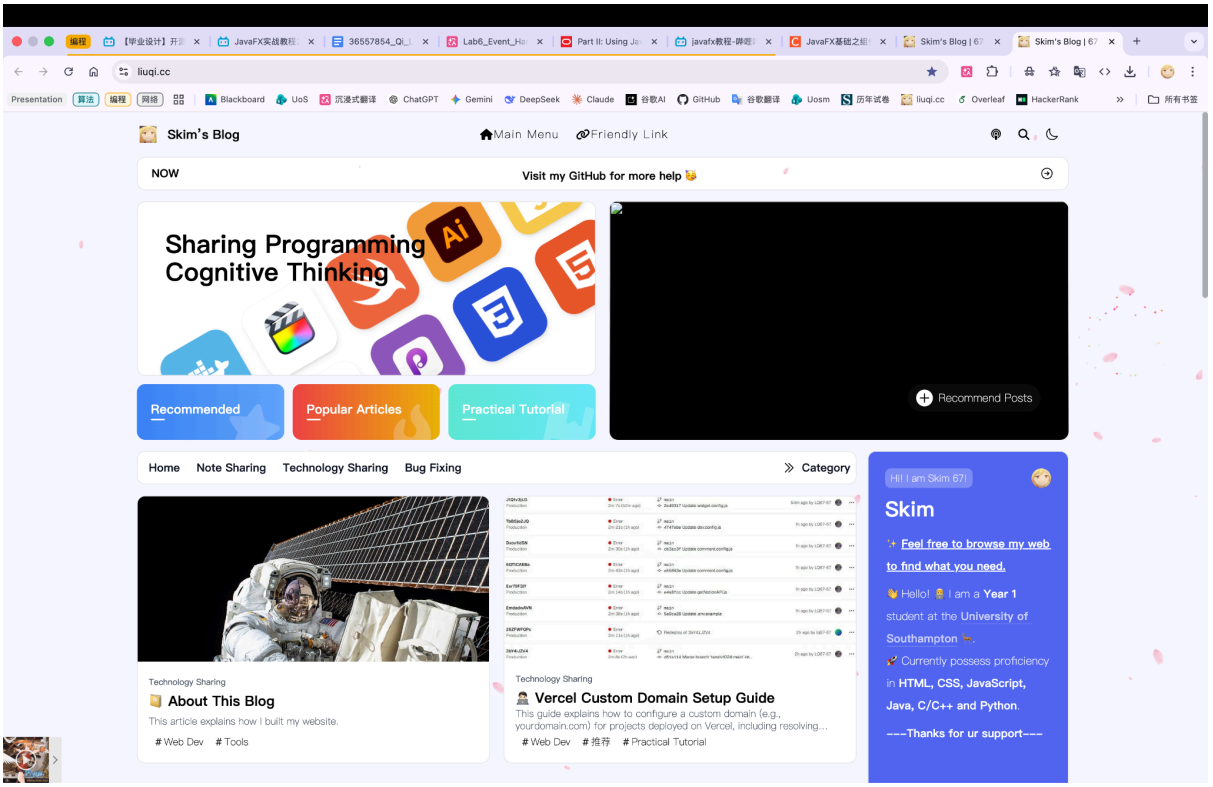
ButtonClickApp:

```
package com.example.lab6;

import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class ButtonClickApp extends Application {
   @Override
   public void start(Stage stage) {
       Label label = new Label("Click the button!");

       Button button1 = new Button("Click Me");
       button1.setOnAction(e -> label.setText("Button Clicked!")); // e is
actually event handler which is lambda expression

       Button button2 = new Button("Click Me :)");
       button2.setOnAction(e -> label.setText("Button Clicked! I am Button
2"));
       button2.setOnMouseClicked(e -> label.setText("Button Clicked!")); //
same way like above but this is mouse click event handler

       button2.setOnAction(e -> {
           getHostServices().showDocument("https://www.liuqi.cc"); // get
access to host services and open the website in default browser
       });

       VBox vbox = new VBox(10, label, button1,button2);

       Scene scene = new Scene(vbox, 300, 200);

       stage.setTitle("Event Handling Example");
       stage.setScene(scene);
       stage.getIcons().add(new javafx.scene.image.Image("/icon.png")); //
set the icon of the stage
       stage.show();
   }

   public static void main(String[] args) {
       launch(args);
   }
}
```

Output:



After Click Button 2



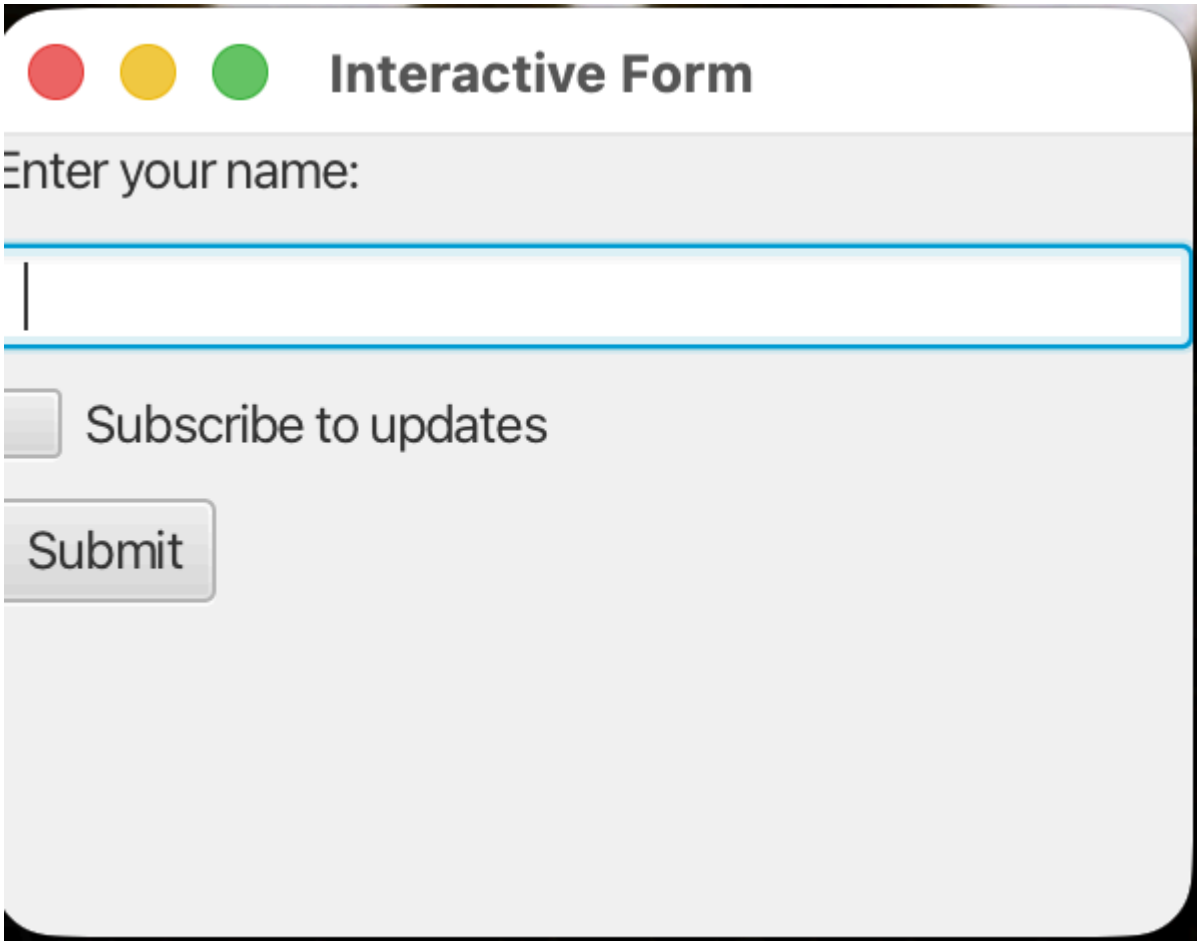
Code:

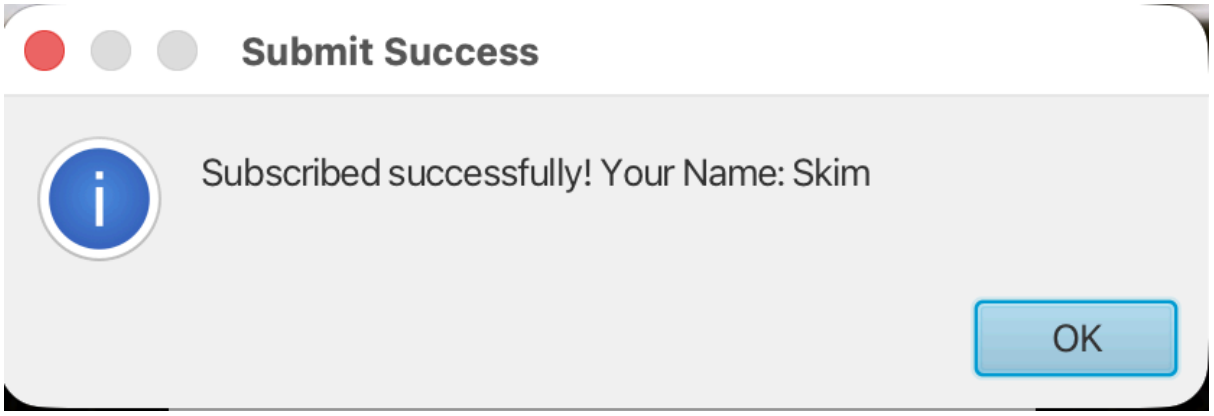
InteractiveFormApp

```
package com.example.lab6;

import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class InteractiveFormApp extends Application {

   @Override
   public void start(Stage stage) {

       Label nameLabel = new Label("Enter your name:");
       TextField nameField = new TextField();
       CheckBox checkBox = new CheckBox("Subscribe to updates");

       Button submitBtn = new Button("Submit");
       submitBtn.setOnAction(e -> {
           String name = nameField.getText(); // nameField is used to get the
text input from user
           boolean subscribed = checkBox.isSelected(); // get the state of
the checkbox

           if (name.isEmpty()) {
               Alert failAlert = new Alert(Alert.AlertType.ERROR);
               failAlert.setTitle("Submit Failed");
               failAlert.setHeaderText(null);
               failAlert.setContentText("Name cannot be empty.");
               failAlert.showAndWait();
               return;
           }

           Alert successAlert = new Alert(Alert.AlertType.INFORMATION);
           successAlert.setTitle("Submit Success");
           successAlert.setHeaderText(null);
           if (subscribed) {
               successAlert.setContentText("Subscribed successfully! Your
Name: " + name);
           } else {
               successAlert.setContentText("Submitted successfully!");
           }
           successAlert.showAndWait();
       });

       VBox vbox = new VBox(10, nameLabel, nameField, checkBox, submitBtn);

       Scene scene = new Scene(vbox, 300, 200);
       stage.setTitle("Interactive Form");
```

```
        stage.setScene(scene);
        stage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Output:



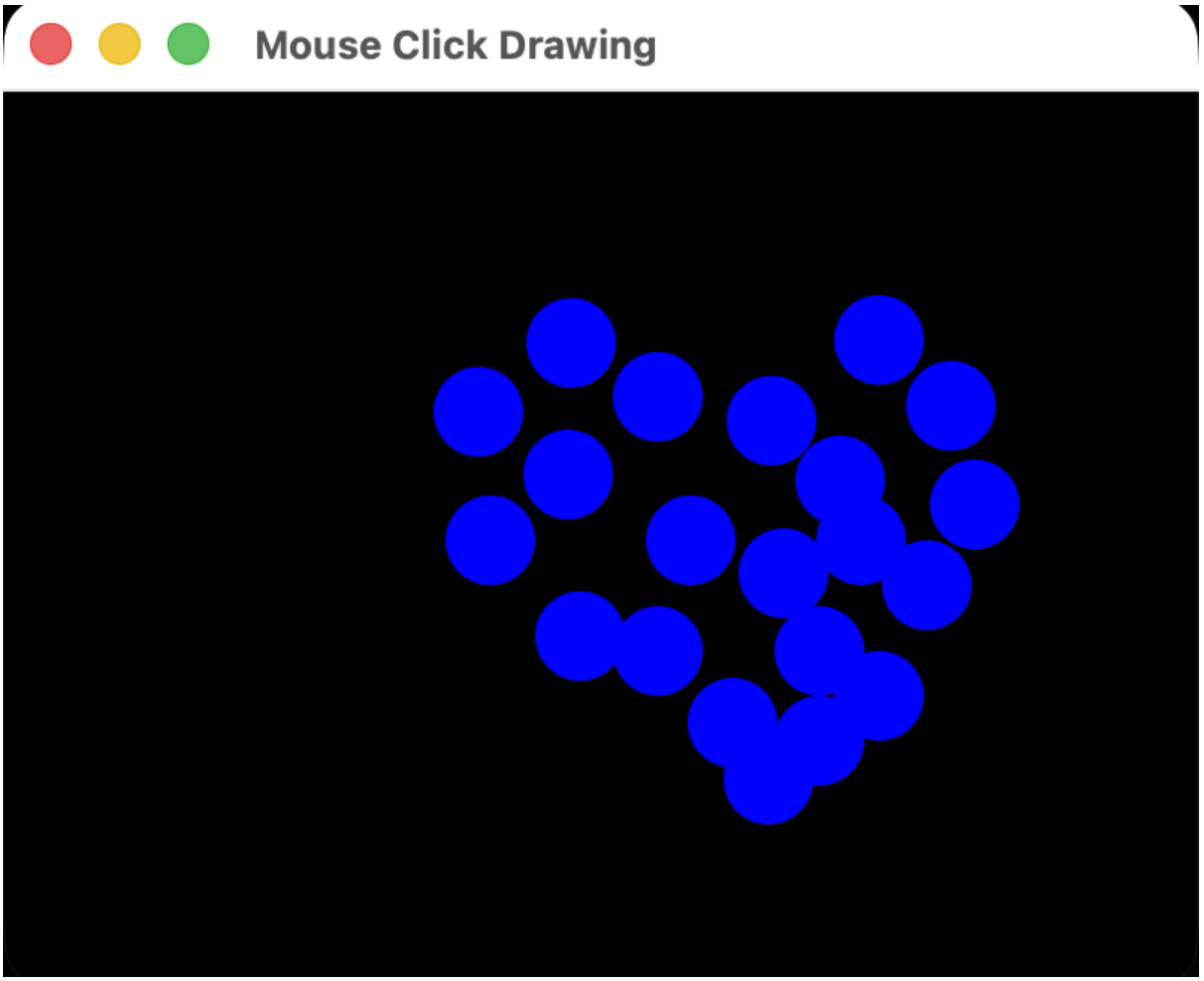


Code:

MouseDrawingApp

```
package com.example.lab6;

import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.canvas.*;
import javafx.scene.layout.StackPane;
import javafx.scene.paint.Color;
import javafx.stage.Stage;

public class MouseDrawingApp extends Application {

   @Override
   public void start(Stage stage) {

       Canvas canvas = new Canvas(400, 300); // Canvas is used to create a
drawing area where can draw shapes, images, or text
       GraphicsContext gc = canvas.getGraphicsContext2D();

       // White background
       gc.setFill(Color.BLACK);
       gc.fillRect(0, 0, 400, 300);

       // Draw blue circle on mouse click instead of setonAction
       canvas.setOnMouseClicked(e -> {
           double x = e.getX(); // used to
           double y = e.getY();
           gc.setFill(Color.BLUE);
           gc.fillOval(x - 15, y - 15, 30, 30);

           /* these codes used to set up red rectangle
            gc.setFill(Color.RED);
            gc.fillRect(x - 15, y - 15, 30, 30);
            */
       });

       StackPane root = new StackPane(canvas);

       Scene scene = new Scene(root, 400, 300);
       stage.setTitle("Mouse Click Drawing");
       stage.setScene(scene);
       stage.show();
   }

   public static void main(String[] args) {
       launch(args);
   }
}
```

Output:



Code:

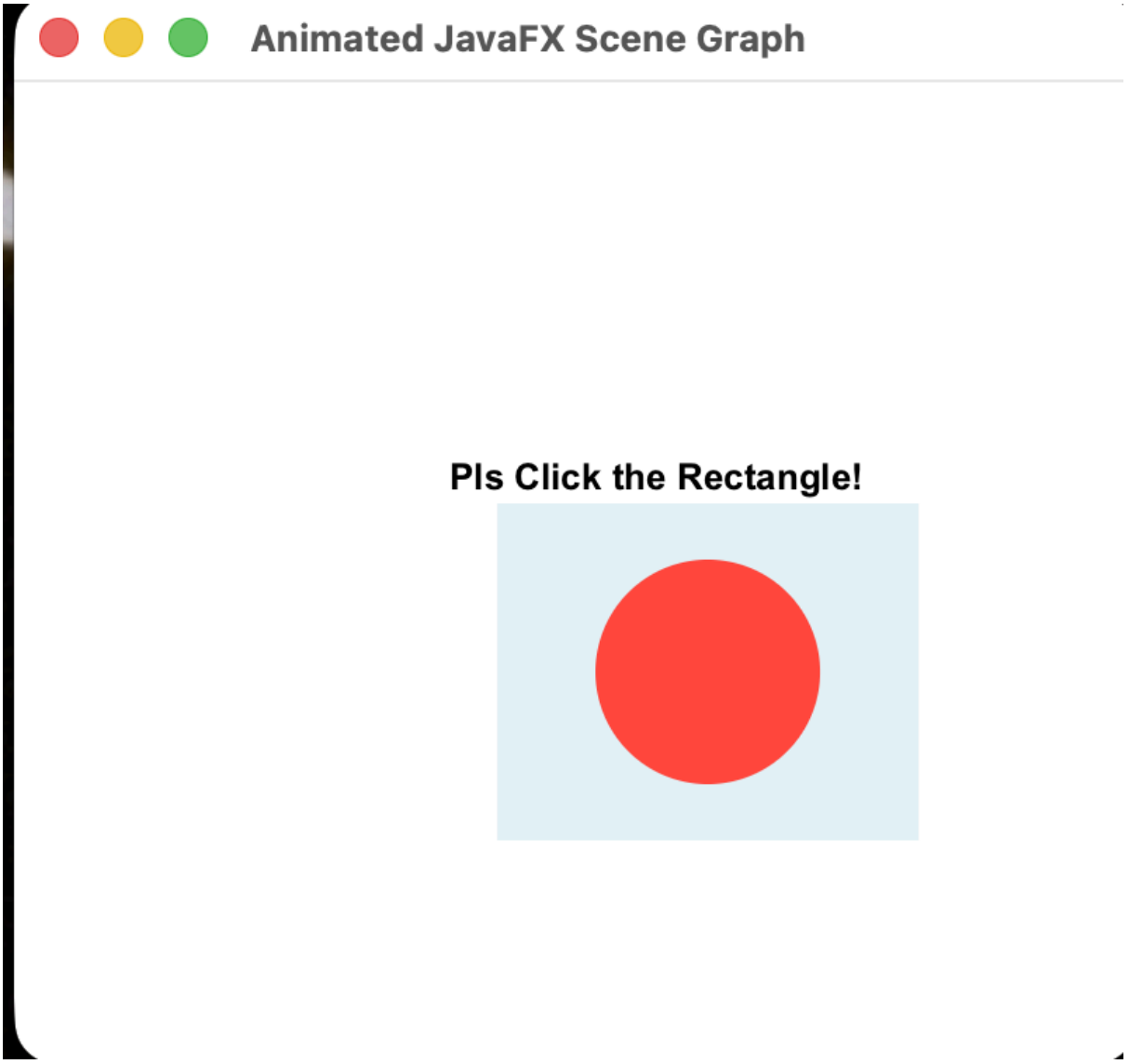
SceneGraphAnimated

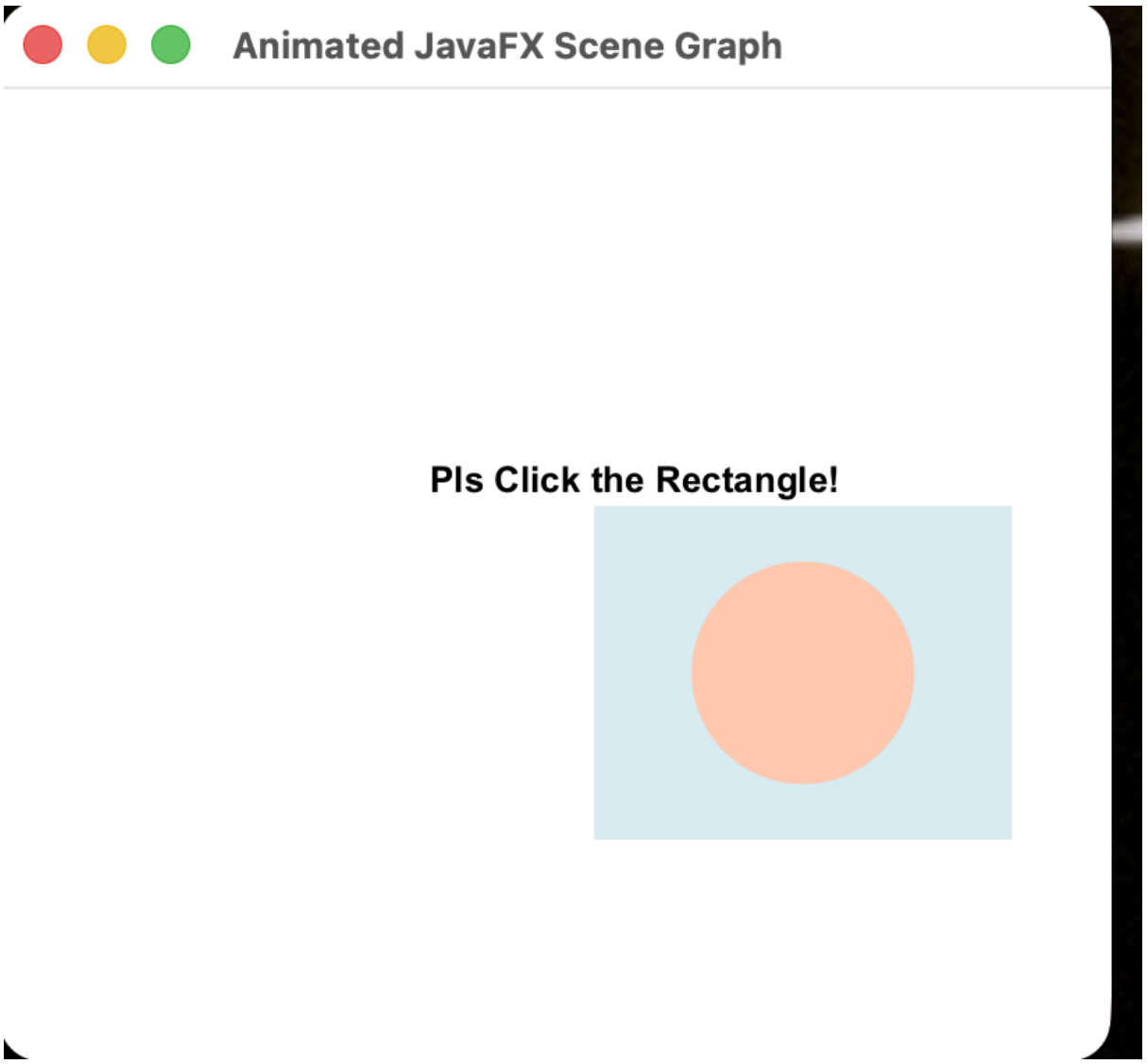
```
package com.example.lab6;

import javafx.animation.*;
import javafx.application.Application;
import javafx.scene.*;
import javafx.scene.paint.Color;
import javafx.scene.shape.*;
import javafx.scene.text.*;
import javafx.stage.Stage;
import javafx.util.Duration;

public class SceneGraphAnimated extends Application {

    @Override
    public void start(Stage stage) {
```

```
        Rectangle rect = new Rectangle(150, 150, 150, 120);
        rect.setFill(Color.LIGHTBLUE);
        rect.setOnMouseClicked(e -> rect.setRotate(rect.getRotate() + 10));

        Circle circle = new Circle(225, 210, 40, Color.RED);

        Text text = new Text(155, 145, "Pls Click the Rectangle!");
        text.setFont(Font.font("Arial", FontWeight.BOLD, 13));

        // Move circle left and right
        TranslateTransition move = new
TranslateTransition(Duration.seconds(1), circle);
        move.setByX(80); // setByX is used to specify how much to move the
circle in x direction
        // move.setByY(80); // setByY: how much to move the circle in y
direction
        move.setAutoReverse(true); // setAutoReverse is used to automatically
reverse the animation back to the original position after it finishes
        move.setCycleCount(Animation.INDEFINITE);
        move.play();

        // Rectangle follows the circle movement
        TranslateTransition rectMove = new
TranslateTransition(Duration.seconds(1), rect);
        rectMove.setByX(80);
        rectMove.setAutoReverse(true);
        rectMove.setCycleCount(Animation.INDEFINITE);
        rectMove.play();

        // Change circle color red to yellow
        FillTransition colorChange = new FillTransition(Duration.seconds(1),
circle);
        colorChange.setFromValue(Color.RED);
        colorChange.setToValue(Color.LIGHTYELLOW);
        colorChange.setAutoReverse(true);
        colorChange.setCycleCount(Animation.INDEFINITE);
        colorChange.play();

        // Blink effect for rectangle (opacity 1 -> 0 -> 1)
        FadeTransition blink = new FadeTransition(Duration.seconds(0.5),
rect);
        blink.setFromValue(1.0);
        blink.setToValue(0.0);
        blink.setAutoReverse(true);
        blink.setCycleCount(Animation.INDEFINITE);
        blink.play();

        Group root = new Group(rect, circle, text);

        Scene scene = new Scene(root, 400, 350, Color.WHITE);
        stage.setTitle("Animated JavaFX Scene Graph");
        stage.setScene(scene);
        stage.show();
```

```
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Output:




Animated JavaFX Scene Graph
Pls Click the Rectangle!